

PROJECT PROPOSAL

Scalable Relational Architecture for AI-Assisted Healthcare

CareSync Advanced DBMS

PROJECT TEAM

- Muhammad Ehtisham Hashim
- Muhammad Abdullah Siddique
- Raja Muhammad Ali
- Ali Ahmad

DOCUMENT DETAILS

Date: July 6, 2026

Version: v1.0 (Final)

Prepared For: Ma'am Shahida

1. Introduction & Problem Statement

Project Overview

CareSync is a state-of-the-art healthcare management platform designed to modernize clinical interactions. The system pioneers a hybrid data architecture by tightly coupling strict relational clinical records with unstructured multimedia storage (audio transcripts) and dynamic AI integration (Google Gemini) for automated SOAP note generation.

Problem Statement

Traditional clinical databases are rigidly structured and poorly equipped to handle modern data formats like voice recordings and AI-generated text. Consequently, healthcare providers suffer from severe data fragmentation, lethargic query performance during complex historical patient lookups, and transaction bottlenecks caused by concurrent scheduling conflicts.

CORE OBJECTIVES

- Engineer a rigid relational schema capable of seamlessly integrating unstructured JSON data.
- Enforce strict data consistency and eliminate write anomalies across multi-doctor facilities.
- Optimize query execution pathways to aggregate longitudinal patient histories instantly.
- Automate HIPAA-level compliance via immutable, database-level audit triggers.

2. Requirements Analysis

FUNCTIONAL REQUIREMENTS

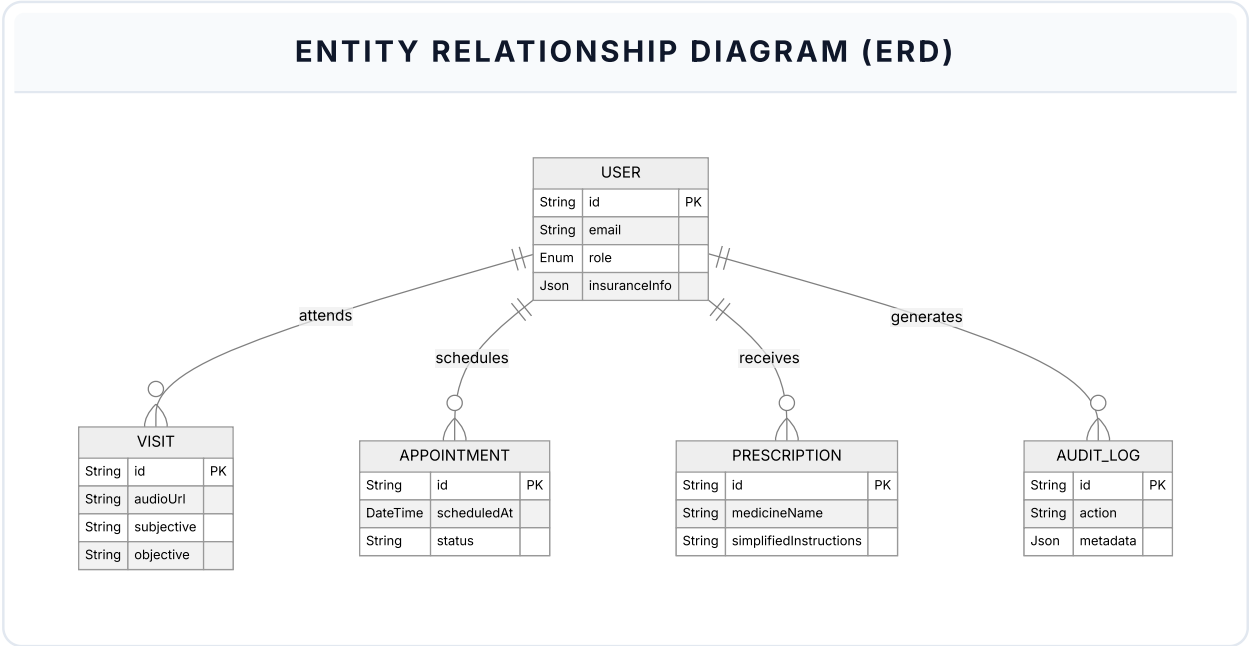
- › **Role-Based Access:** Strict workflow segregation for PATIENT, DOCTOR, and ADMIN profiles.
- › **Appointment Logic:** Scheduling algorithms with built-in conflict resolution.
- › **Clinical Records:** Centralized storage for vital signs, prescriptions, and AI-generated SOAP notes.
- › **Audit Trails:** Automatic generation of immutable logs for all data mutations.

NON-FUNCTIONAL REQUIREMENTS

- › **Performance Benchmark:** Transaction response times strictly under 500ms.
- › **High Availability:** 99.9% uptime architecture using edge-cached cloud infrastructure.
- › **Cryptographic Security:** All endpoints protected by JWT authentication and bcrypt hashing.
- › **Scalability:** Advanced connection pooling designed to handle sudden concurrent traffic spikes safely.

3. Database Design & Architecture

The architecture revolves around the **User** entity (differentiated by Role), tracking interactions through **Appointments** and **Visits**. A Visit binds unstructured audio interactions with AI-generated SOAP notes, relating directly to medical **Prescriptions**. Every data modification instantly triggers an entry in the **AuditLog**.



Relational Schema & Normalization Strategy

The database is rigorously normalized to **Boyce-Codd Normal Form (BCNF)**. Multi-valued attributes like patient vitals and allergies have been decoupled into distinct `Vital` and `Allergy` tables referencing the `User` via Foreign Keys. Transitive dependencies are eliminated to categorically prevent update anomalies. Semi-structured, highly variable data (e.g., unstructured lab results) securely utilizes PostgreSQL's optimized `JSONB` datatype to maintain schema flexibility without sacrificing relational integrity.

4. Advanced Implementation Details

Technology Stack

The core engine is **PostgreSQL (NeonDB)**, strictly managed via the **Prisma ORM** to guarantee type safety. **Upstash Redis** serves as a high-speed caching and rate-limiting broker. **Supabase** provides secure object storage exclusively for clinical audio recordings.

Advanced SQL Components

Indexing Strategies: B-Tree indexing is strictly enforced on all Foreign Keys (e.g., `patientId`) yielding $O(\log N)$ join speeds. Unique Hash constraints optimize high-traffic lookup attributes like emails.

Triggers & Procedures: Automated, immutable database triggers enforce the instantiation of `AuditLog` entries immediately upon critical table mutations.

Referential Integrity: Explicit cascading deletes guarantee no orphan records exist when dependent entities are purged from the system.

Transaction Management

We leverage PostgreSQL's robust **Multi-Version Concurrency Control (MVCC)**. Critical write paths—such as scheduling identical doctor appointment slots—are protected by transactions operating under the Read Committed isolation level, effectively mitigating dirty reads and concurrent race conditions.

5. Query Optimization & Testing Plan

A Sample Complex Queries

- **Longitudinal Patient Aggregation:** Complex multi-table JOIN operations combining the User, Visit, Prescription, and Vital tables to construct a complete medical history timeline using a singular, highly-optimized query execution plan.
- **Analytical Subqueries:** Deploying Window Functions and grouped aggregations over the Appointment table to pinpoint high-load doctors and peak clinical hours for predictive scheduling algorithms.

B Testing Strategy

- **Mock Data Population:** Development pipelines utilize deterministic database seed scripts to inject thousands of synthesized patient records, appointments, and medical histories.
- **Constraint Validation:** Rigorous automated integration testing designed specifically to attempt constraint violations (e.g., overlapping appointment times, unauthorized foreign key mutations) to cryptographically verify schema integrity.